

A

玻璃罩要框住所有的试剂，但是在罩子的边界不视为在罩内。所以先计算出最大和最小的 x 坐标和 y 坐标，然后将最小坐标 -1 ，最大坐标 $+1$ 即为罩子四个点的坐标。

```
1  #include<bits/stdc++.h>
2  #define int long long
3  using namespace std;
4
5  signed main()
6  {
7      int n;
8      cin>>n;
9      int mxx=-1e9, mxy=-1e9, minx=1e9, miny=1e9;
10     for(int i=1;i<=n;i++)
11     {
12         int x, y;
13         cin>>x>>y;
14         mxx=max(mxx,x);
15         minx=min(minx,x);
16         mxy=max(mxy,y);
17         miny=min(miny,y);
18     }
19     cout<<minx-1<<" "<<miny-1<<"\n"<<mxx+1<<" "<<mxy+1;
20 }
```

B

注意到数据范围很小，因此从每个格子开始向八个方向搜索，搜索时记录一下有没有转弯即可，主要难点在于代码的实现

```
1  #include<bits/stdc++.h>
2  using namespace std;
3
4  const int x[]={1,0,-1,0};
5  const int y[]={0,1,0,-1};
6  const int x2[]={1,-1,-1,1};
7  const int y2[]={1,1,-1,-1};
8  string s;
9  int n,m,ans;
10 char a[105][105];
11
12 void dfs(int dep,int x,int y,int hd,bool f=0)
13 //f=0表示还没有转弯
14 //f=1表示已经转弯过了
15 {
16     if(dep==s.size())
17     {
18         //cerr<<x<<' '<<y<<endl;
19         ans++;
20         return;
21     }
22     //不转弯
```

```

23 //转弯
24 int x_=x+X[hd],y_=y+Y[hd];
25 if(x_>0&&y_>0&&x_<=n&&y_<=m&&a[x_][y_]==s[dep]) dfs(dep+1,x_,y_,hd,f);//
    妹 e 父壁帮絳涓嬧悒
26 x_=x+X[(hd+1)%4];
27 y_=y+Y[(hd+1)%4];
28 if(x_>0&&y_>0&&x_<=n&&y_<=m&&dep>1&&!f&&a[x_][y_]==s[dep])
dfs(dep+1,x_,y_,(hd+1)%4,1);//鑒宠漿 90擄鐔帆笰錫?
29 x_=x+X[(hd+3)%4];
30 y_=y+Y[(hd+3)%4];
31 if(x_>0&&y_>0&&x_<=n&&y_<=m&&dep>1&&!f&&a[x_][y_]==s[dep])
dfs(dep+1,x_,y_,(hd+3)%4,1);//宸一漿 90擄鐔帆笰錫?
32 }
33
34 void dfs2(int dep,int x,int y,int hd,bool f=0)
35 {
36     if(dep==s.size())
37     {
38         //cerr<<x<<' '<<y<<endl;
39         ans++;
40         return;
41     }
42     int x_=x+X2[hd],y_=y+Y2[hd];
43     if(x_>0&&y_>0&&x_<=n&&y_<=m&&a[x_][y_]==s[dep]) dfs2(dep+1,x_,y_,hd,f);
44     x_=x+X2[(hd+1)%4];
45     y_=y+Y2[(hd+1)%4];
46     if(x_>0&&y_>0&&x_<=n&&y_<=m&&dep>1&&!f&&a[x_][y_]==s[dep])
dfs2(dep+1,x_,y_,(hd+1)%4,1);
47     x_=x+X2[(hd+3)%4];
48     y_=y+Y2[(hd+3)%4];
49     if(x_>0&&y_>0&&x_<=n&&y_<=m&&dep>1&&!f&&a[x_][y_]==s[dep])
dfs2(dep+1,x_,y_,(hd+3)%4,1);
50 }
51
52 int main()
53 {
54     cin>>s>>n>>m;
55     for(int i=1;i<=n;i++)
56     {
57         for(int j=1;j<=m;j++)
58         {
59             cin>>a[i][j];
60         }
61     }
62     for(int i=1;i<=n;i++)
63     {
64         for(int j=1;j<=m;j++)
65         {
66             if(a[i][j]==s[0])
67             {
68                 dfs(1,i,j,0);
69                 dfs(1,i,j,1);
70                 dfs(1,i,j,2);
71                 dfs(1,i,j,3);
72                 dfs2(1,i,j,0);
73                 dfs2(1,i,j,1);

```

```

74         dfs2(1,i,j,2);
75         dfs2(1,i,j,3);
76     }
77 }
78 }
79 cout<<ans;
80 return 0;
81 }

```

C

遍历压缩字符串，解析出每个字符及其对应的重复次数，并计算字符串的总长度 sum 。通过 $c \bmod sum$ 找到第 c 个字符在原始字符串中的位置。再次遍历压缩字符串，累加字符的重复次数，直到找到第 $c \bmod sum$ 个字符。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  long long sum, c, ans;
4  string s;
5  int main() {
6      cin >> s >> c;
7      for (int i = 0; i < s.length(); ) {
8          char ch = s[i++];
9          long long cc = 0;
10         //字符是字母的时候
11         //字符串转十进制
12         //9  cc=9
13         //7  cc=cc*10+7=97
14         //1  cc=cc*10+1=971
15         //2  cc=cc*10+2=9712
16         while (i < s.length() && isdigit(s[i])) {
17             cc = cc * 10 + (s[i] - '0');
18             i++;
19         }
20         sum += cc;
21     }
22     // c=(c-1)%sum+1;
23     //sum刚好是c的倍数的时候,sum->0
24     c %= sum;
25     for (int i = 0; i < s.length(); ) {
26         char ch = s[i++];
27         long long cc = 0;
28         while (i < s.length() && isdigit(s[i])) {
29             cc = cc * 10 + (s[i] - '0');
30             i++;
31         }
32         if (ans + cc > c) {
33             cout << ch << endl;
34             return 0;
35         }
36         ans += cc;
37     }
38     return 0;

```

D

如果已经分好了队，对于每一队来说，最后吃完的人吃完的时间是

$$\max_i \sum_{j=1}^i a_j + b_i$$

一定是按 b 降序排序更优，证明如下：

设 a 的前缀和为 s ，假设最后排序方案存在 (i, j) ，其中 $b_i < b_j, i < j$ ，那么

- i 吃完的时间是 $s_{i-1} + a_i + b_i$
- j 吃完的时间是 $s_{j-1} + a_j + b_j$

如果交换 (i, j) ，则

- i 吃完的时间是 $s_{j-1} + a_i + b_i$
- j 吃完的时间是 $s_{i-1} + a_j + b_j$

容易发现前种情况两者取 $\max$ 不小于后种情况的两者的 $\max$ ，因此交换 (i, j) 一定不劣。

因此，我们将人按照 b 降序排序，设 $f_{i,j}$ 表示考虑前 i 个人，第一队已经分配进去的人 a 的和是 j ，转移如下

$$f_{i,j} = \min(\max(j + b_i, f_{i-1,j}), \max(s_i - j + b_i, f_{i-1,j}))$$

可以去掉第一维，时间复杂度 $O(n^2V)$ 。

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  int n,s,f[250005];
4  struct node{
5      int a,b;
6  }t[505];
7  bool cmp(node x,node y){
8      return x.b>y.b;
9  }
10 int main(){
11     cin>>n;
12     for(int i=1;i<=n;i++)cin>>t[i].a>>t[i].b;
13     sort(t+1,t+n+1,cmp);
14     memset(f,0x3f,sizeof(f));
15     f[0]=0;
16     for(int i=1;i<=n;i++){
17         s+=t[i].a;
18         for(int j=s;j>=0;j--){
19             f[j]=max(f[j],s-j+t[i].b);
20             if(j>=t[i].a)f[j]=min(f[j],max(j+t[i].b,f[j-t[i].a]));
21         }
22     }
23     int ans=1e9;
24     for(int i=0;i<=s;i++)ans=min(ans,f[i]);
25     cout<<ans;
26 }
```

E

30pts

2^N 枚举所有情况进行判定，时间复杂度为 $O(2^N)$

+30pts

对于 $0 \leq a_i \leq 3$, $f[i][j][k]$ 表示考虑了序列的前 i 个数，其中 A 集中所有元素的或运算为 j , B 集中所有元素的或运算为 k 的方案数，其中 $j, k \in [0, 3]$, 背包进行转移，时间复杂度为 $O(N)$ 。

100pts

对于二进制下某一位 k , 若满足第 k 位上为 1 的 a_i 个数不为 0 个，那么所有第 k 位上为 1 的数不能划分在同一个集合。

例如对于 $\{(101)_2, (100)_2, (001)_2, (111)_2\}$ 这三个数， $\{(101)_2, (001)_2, (111)_2\}$ 一定不能被划分在同一个集合，否则另一个集合的最低位一定为 0，不满足题设条件。

所有第 k 位上为 1 的数不能划分在同一个集合并不好处理，我们考虑其相反面进行容斥，所有第 k 位上为 1 的数都应当划分在同一个集合并不好处理

考虑容斥，强制钦定第 k 位上为 1 的数划分在同一个集合。

现在有若干数强制钦定在同一个集合，将这些数用并查集连接，在同一个并查集里的数说明一定要被分配在一个集合里，考虑乘法原理，每个并查集的联通块都有 2 种可能，即划分在 A 或 B ，那么方案数即为 $2^{\text{并查集联通块个数}}$ ，

容斥一共有 $O(2^{15})$ 种情况。

对于每一种情况，需要进行 $O(15 \times N)$ 的判断，但所有数最多合并到同一个并查集里，并查集部分的复杂度为 $N \times \alpha(N)$ ，复杂度为 $O(2^{15} \times (15 \times N + N \times \alpha(N)))$ 。

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  const int N=500+7,mod=998244353;
4  int f[N],a[N];
5  int g(int u){
6      if(f[u]==u) return u; return f[u]=g(f[u]);
7  }
8  int pows(long long u,int v){
9      long long ans=1;
10     while(v>0){
11         if(v&1) ans=ans*u%mod; u=u*u%mod,v=v>>1;
12     }
13     return ans;
14 }
15 int main(){
16     int n,ans=0,D=0;
17     cin>>n;
18     for(int i=1;i<=n;i++) cin>>a[i],D|=a[i];
19     //33=2^5+2^0
20     //2^1
21     for(int j=0;j<(1<<15);j++){
22         if((D&j)!=j) continue;
23         for(int k=1;k<=n;k++) f[k]=k;//并查集初始化
```

```

24     for(int k=0;k<=14;k++){
25         if(!(j&(1<<k))) continue;
26         //(1<<k)这一位上为1
27         //所有这一位上为1的数都要放到一个并查集里
28         int j=0;
29         //所有满足 (1<<k)这一位上为1的数里面的第一个值
30         for(int c=1;c<=n;c++){
31             if(a[c]&(1<<k)){
32                 if(!j) j=c;
33                 f[g(j)]=g(c);
34                 //j,c放到一个并查集里
35             }
36         }
37     }
38     int s=0;
39     for(int k=1;k<=n;k++) if(g(k)==k) s++;
40     //求解j在二进制下有多少个1
41     // +1
42     // -1
43     if(__builtin_popcount(j)%2==0) ans=(ans+pows(2,s))%mod;
44     else ans=(ans-pows(2,s)+mod)%mod;
45 }
46 cout<<ans<<endl;
47 return 0;
48 }
49

```